

Technical Report · Smart Vehicular Systems

Project 5: Roadside Assistant for Hidden Pedestrians and Cyclists

An infrastructure-to-vehicle warning system over MQTT,
with YOLOv8 detection in CARLA

Authors

Shkelzen Tafili	shkelzen.tafili@studio.unibo.it
Dias Katrenov	dias.katrenov@studio.unibo.it
Danial Khayatian	danial.khayatian@studio.unibo.it

CONTENTS

Executive Summary	2
1 Introduction	3
1.1 The Project	3
1.2 Motivation	3
1.3 Report Structure	3
2 Requirements	4
3 Design of the Solution	5
3.1 Architecture and Processing Loop	5
3.2 Scenario Layout and RSU Placement	5
3.3 Detection with YOLOv8n	6
3.4 From Pixels to World Coordinates	7
3.5 Tracking	7
3.6 Warning Message Design	7
3.7 MQTT Transport and Shared Timestamps	8
3.8 Path Check: Will the Road User Be in My Way?	9
3.9 Fusion and Slow-Down Policy	9
3.10 Alert Display	9
3.11 Driving	11
4 Technologies Used	12
5 Testing and Results	13
5.1 Experimental Design	13
5.2 Metrics	13
5.3 Results	14
5.4 MQTT Link Performance	16
5.5 Limitations of the Evaluation	17
6 Conclusions	18
6.1 Summary	18
6.2 Limitations	18
6.3 Future Work	18
Bibliography	19

EXECUTIVE SUMMARY

A car approaching a crossing often cannot see a pedestrian or cyclist who is about to step out from behind a parked vehicle. This project builds and evaluates a **roadside assistant** for exactly that situation in the CARLA simulator. A roadside unit (RSU) with a pole-mounted camera watches the crossing, detects pedestrians and cyclists with the **YOLOv8n** detector, converts each detection into **world coordinates**, and sends the result to the approaching car as a **compact warning message** (about 120 bytes per object), published over **MQTT**, with an emulated 100 ms radio transmission delay. Both sides use CARLA's simulation clock as a **shared time base**. The car combines the RSU information with its own camera, checks whether each road user will actually be in its path, shows a warning, and slows down or brakes.

Three occlusion scenarios were run, each once with the car's own camera only (baseline) and once with the RSU warning (assisted): a pedestrian behind a parked van, a cyclist behind a parked truck, and a group of 15 pedestrians behind the van in heavy rain. **Without the RSU warning, all three ended in a collision or a near miss** (minimum gaps of 1.58 m, 0.00 m and 0.76 m); in two of them the car's camera never saw the road user in time to react. **With the RSU warning, all three ended safely**, with 2.8 to 7.6 m to spare. The RSU warning reached the car 1.2 to 2.4 s (10 to 19 m) before it would have reached the crossing. The measured end-to-end message latency was 160 ms (of which about 1.5 ms MQTT delivery), no messages were lost, and the RSU's position estimates were within 0.75 m of the true position for pedestrians and 2.1 m for the cyclist.

CHAPTER 1: INTRODUCTION

1.1 The Project

The task is to place a virtual roadside camera at a crossing and use it to warn a vehicle about a hidden pedestrian or cyclist. The camera must cover the part of the crossing the vehicle cannot see; its detections must be converted into a compact message and sent to the vehicle; the vehicle must fuse this warning with its own perception, display a credible alert and apply a simple slow-down policy. Behaviour with and without assistance is compared in at least two occlusion scenarios, measuring warning lead time, speed reduction and avoided near misses.

Everything runs in CARLA [1] through its Python API, in a single notebook. The system has three layers, mirroring a real deployment: **infrastructure perception** (the RSU camera, the detector and the conversion to world coordinates), **communication** (the warning message, its MQTT transport and the shared clock), and **vehicle decision and control** (path check, fusion, policy, alert and driving).

1.2 Motivation

- Occlusion by parked vehicles is a classic cause of pedestrian and cyclist accidents: by the time the road user is visible to the driver or the car's sensors, there may be too little distance left to stop.
- An elevated roadside camera has a view that no on-board sensor can have, and it can see the road user several seconds before the car can.
- Communication from the infrastructure to the vehicle makes that view available to the car, but only with a delay; a useful design has to work with slightly outdated information.

1.3 Report Structure

Chapter 2 maps the project requirements to the implementation. Chapter 3 describes the design: the scenario, the RSU camera, detection, pixel-to-world conversion, tracking, the warning message and its MQTT link, and the vehicle-side logic. Chapter 4 lists the technologies. Chapter 5 presents the experiments and the measured results, and Chapter 6 concludes with strengths, limitations and future work.

CHAPTER 2: REQUIREMENTS

The table maps each point of the project description to the part of the implementation that addresses it. The messaging requirement (“MQTT/V2X”) is met with MQTT: the RSU publishes its warning messages to an MQTT topic to which the vehicle subscribes.

Project requirement	Implementation
Roadside camera covering the hidden part of the crossing	RGB camera on a 5 m pole, pose chosen by projecting the relevant points into its image (Section 3.2)
Detector for pedestrians and cyclists	YOLOv8n, person and bicycle classes; a person on a bicycle is merged into a cyclist (Section 3.3)
World coordinates	Back-projection of the foot point of each detection onto the road plane (Section 3.4)
Compact message sent to the vehicle	JSON warning message, 53 bytes + about 71 bytes per object (Section 3.6)
MQTT/V2X messaging	MQTT publish/subscribe through a local MQTT broker, plus an emulated 100 ms radio delay and optional packet loss (Section 3.7)
Shared timestamps	Capture time on CARLA’s simulation clock in every message; used for delay, latency measurement and age compensation (Section 3.7)
Warning receiver and slow-down or alert policy	Receiver, path check, CAUTION / DANGER policy (Sections 3.8, 3.9)
Fuse the external warning with local perception	RSU and car-camera objects go through the same path check; the policy combines them (Section 3.9)
Credible alert	Banner with level, type, distance, source and message age, plus a marker on the hidden road user (Section 3.10)
Compare with and without assistance in at least two occlusion scenarios	Three scenarios, each run as baseline and assisted (Chapter 5)
Measure warning lead time, speed reduction, avoided near misses	Computed per run and summarised in a comparison table (Sections 5.2, 5.3)

Table 2.1: Project requirements and their implementation.

Non-functional requirements: every run must be **repeatable** (synchronous simulation, a deterministic driver, fixed start times), the detector must be usable **without a GPU**, and each run must **clean up** its actors and restore the server settings even after an error.

CHAPTER 3: DESIGN OF THE SOLUTION

3.1 Architecture and Processing Loop

The simulation runs in **synchronous mode** with a fixed step of 0.04 s (25 Hz): the world only advances when the client calls `world.tick()`, so the results do not depend on how fast the computer is. Both cameras deliver a 720×480 image every 0.2 s. On every detection cycle and every tick the following happens:

Step	Side	What happens
1	RSU (every 0.2 s)	YOLOv8n detects pedestrians and cyclists; each detection is converted to world coordinates; the tracker assigns an ID and a velocity; the tracks are encoded into a warning message stamped with the capture time and sent
2	Link (MQTT)	The message is published to an MQTT topic; the car's receiver holds it until capture time + 100 ms on the shared simulation clock (emulated radio delay)
3	Vehicle (every 0.2 s)	The car's own camera runs the same detection, conversion and tracking
4	Vehicle (every tick)	Delivered RSU objects are moved forward to the current time; every object is checked against the car's path; the policy decides; the driver applies throttle, brake and steering; the alert is drawn
5	Evaluation (every tick)	CARLA's true positions are used only to measure the gap between the car and the road users

Table 3.1: Processing loop.

3.2 Scenario Layout and RSU Placement

The scenario is set in CARLA's Town03. The ego car (Tesla Model 3) starts at (103, 133) and drives towards a crossing at $x = 130.5$. A parked vehicle (a Mercedes Sprinter van, or a truck in the cyclist scenario) stands at the kerb just before the crossing. Pedestrians and the cyclist start on the pavement at $y = 145$ to 146 and cross the road in the negative y direction (Figure 3.1). While the car approaches, the parked vehicle hides the last metres of their path; they become visible to the car's camera only when they step into the lane, often when the car is already level with them.

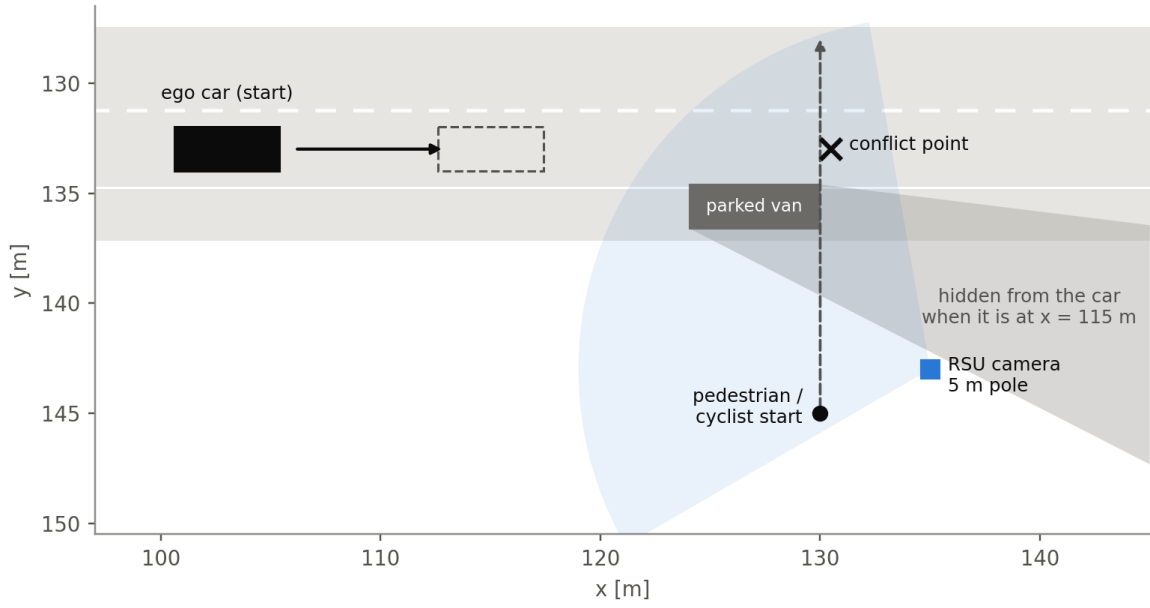


Figure 3.1: Top view of the scenario (actor positions from the notebook; road edges approximate). The blue sector is the RSU camera’s horizontal field of view; the grey area is hidden from the car by the van when the car is at $x = 115$ m.

The RSU camera is mounted on a 5 m pole at (135, 143), pitched 35° down and turned to a yaw of -155° , with a 110° field of view. The pose was chosen by projecting the key points of the scene into the camera image and requiring all of them to lie at least 75 pixels inside the image: the start positions of the pedestrian and the cyclist, the area behind the van, both lanes at the crossing and the car’s approach. An earlier, lower pose (2.5 m, pitch -10° , yaw -110°) had the start positions behind the camera, so pedestrians were detected only after walking about 3 m; this delayed every warning. At the start of each run the notebook repeats the coverage check and reports road users that start outside the image (only the back rows of the 15-person group in Scenario 3).

3.3 Detection with YOLOv8n

Both cameras use **YOLOv8n** [3], the smallest model of the Ultralytics YOLOv8 family, pre-trained on the COCO dataset [4]. YOLO [2] is a single-stage detector: one forward pass predicts boxes, confidence scores and classes for the whole image, which makes it fast enough to run on a CPU at the 5 Hz detection rate. Only two COCO classes are kept, *person* (0) and *bicycle* (1); all other classes are discarded, so parked vehicles can never be mistaken for pedestrians. A cyclist appears to YOLO as a person above a bicycle; the two boxes are paired and reported as one **cyclist**, and persons without a bicycle are reported as **pedestrians**.

Setting	Car camera	RSU camera	Reason
Person confidence	0.60	0.35	The RSU sees people from 5 m above; they look foreshortened and YOLO's confidence is lower
Bicycle confidence	0.40	0.30	Bicycles are thin objects with lower scores
Effect of a false detection	hard braking	slow-down warning (only if in the car's path)	Justifies the lower RSU threshold

Table 3.2: Detection confidence thresholds.

With the car camera's stricter thresholds, the RSU detected the pedestrian in only about one message in five during early tests (judging by the message sizes), and the warning came late or not at all. Lowering the RSU thresholds, together with the tracker changes in Section 3.5, solved this.

3.4 From Pixels to World Coordinates

A detection box is given in pixels; the car needs a position on the road. Because each camera's pose is known, a pixel can be turned into a point on the ground:

```

f      = W / (2 * tan(FOV / 2))      # focal length in pixels
d_cam  = (1, (u - W/2) / f, -(v - H/2) / f) # ray through pixel (u, v)
d_world = R_cam * d_cam              # rotate into world axes
t      = (z_ground - o_cam.z) / d_world.z # distance to the road plane
p      = o_cam + t * d_world          # point on the road: (x, y)

```

The pixel used is the **bottom centre of the box**, where the feet touch the ground. The rotation R_{cam} and origin o_{cam} come from the camera transform recorded with each image, using the same 4×4 matrix convention as CARLA. For the RSU the ground height is taken at the crossing; for the car's camera it is the road under the car itself (camera height minus mounting height), which is what a real vehicle would assume. The inverse projection (world to pixel) is used to draw hidden road users into the car's camera image. The accuracy is measured in every run by comparing the RSU's estimates with CARLA's true positions (Section 5.4).

3.5 Tracking

A single detection says where someone is, not where they are going. A nearest-neighbour tracker in world coordinates links detections from consecutive cycles, giving each road user a stable **ID** and a **velocity** (the displacement since the last sighting divided by the elapsed time, smoothed with a factor of 0.5). Two features make it robust to missed detections:

- **Predicted association:** each new detection is compared with where each track should be by now (last position plus velocity times elapsed time), and the allowed distance grows with the time since the track was last seen (2.0 m + 2.5 m per second). A person missed for a few frames keeps the same ID and velocity.
- **Coasting:** a confirmed track (seen at least twice) that is missed in a frame is still reported at its predicted position for up to 0.6 s. A single missed detection therefore does not make the warning disappear. Tracks not seen for 1 s are deleted.

The same tracker runs on the RSU (on its detections, before sending) and in the car (on its own camera's detections).

3.6 Warning Message Design

Each RSU detection cycle produces one message, whether or not anything was detected, so the car also learns that the crossing is clear. The message is JSON with short keys and rounded values:

```
{
  "v": 1, "rsu": "RSU-1", "seq": 42, "t": 12.4,
  "objs": [
    { "id": 3, "c": "ped", "x": 130.82, "y": 141.2, "vx": 0.02, "vy": -3.41, "cf": 0.91 }
  ]
}
```

Field	Meaning	Resolution
v	Message format version	integer
rsu	Sender ID	string
seq	Sequence number; gaps reveal lost messages	integer
t	Capture time of the camera frame on the shared simulation clock	1 ms
objs[].id	Track ID, stable across messages	integer
objs[].c	Class: ped (pedestrian) or cyc (cyclist)	string
objs[].x, y	Position in CARLA world coordinates [m]	1 cm
objs[].vx, vy	Velocity [m/s]	0.01 m/s
objs[].cf	Detector confidence	0.01

Table 3.3: Fields of the warning message.

An empty message is 53 bytes and each object adds about 71 bytes: 123 bytes for one pedestrian and about 1.1 kB for 15. At 5 messages per second this is well within the capacity of an MQTT link or a real V2X radio. The content deliberately mirrors the **ETSI Collective Perception Message (CPM)** [5], in which a station shares the objects it has detected, with position, velocity and a time reference. Sending tracked objects with velocities, rather than a simple “pedestrian detected” flag, lets the car decide for itself whether a road user is relevant to its own path, and compensate for the message’s age.

3.7 MQTT Transport and Shared Timestamps

The messages are carried over **MQTT**, a lightweight publish/subscribe protocol. The RSU and the car are two separate MQTT clients (paho-mqtt) connected to a broker (aMQTT, a Python MQTT broker) running on the same computer: the RSU **publishes** each serialized message to the topic `rsu1/vru_warnings` (QoS 0, fire-and-forget, like a radio broadcast), and the car **subscribes** to that topic. Publish/subscribe matches the broadcast nature of a roadside warning: the RSU does not need to know which or how many vehicles listen. If no broker is reachable, the notebook falls back to an in-memory queue carrying the same bytes, so it also runs without MQTT.

MQTT delivery on one computer takes only about 1.5 ms, far faster than a real radio link. The car’s receiver therefore adds an **emulated radio delay**: it releases a message only when the simulation clock has reached the capture time plus **100 ms**, a conservative value for processing plus transmission. Packet loss can be switched on with a configurable probability (it was off for the reported runs). In a real deployment the same content would be broadcast over ITS-G5 or C-V2X, for example as an ETSI Collective Perception Message. No V2X radio is simulated: MQTT is the only communication channel, and CARLA’s V2X sensor was not used, since the assignment allowed either MQTT or V2X.

Both sides read the **same clock**, CARLA’s simulation time. The RSU stamps each message with the time its camera captured the frame, and the car uses that stamp in four ways:

- **Emulated delay:** a message is used only when $sim_now \geq t + 100\text{ ms}$, so the car never acts on information earlier than it could have arrived.
- **Measured latency:** $sim_now - t$ at the moment of use, recorded for every message. It includes the age of the camera frame when it was processed, so it is larger than the 100 ms link delay.
- **Age compensation:** each reported object is moved forward by $velocity \times message\ age$ before it is used, so the car reasons about where the road user is now, not where it was.
- **Freshness:** objects older than 1 s are discarded, so the car never acts on stale information.

3.8 Path Check: Will the Road User Be in My Way?

Not every detected person is a threat: people standing on the pavement, walking away, or who will have finished crossing before the car arrives must not cause braking. Every object, from the RSU or the car's own camera, is converted into the car's frame (distance ahead, sideways offset from the path) and two time windows are compared:

- **Road-user window:** from its sideways velocity, when it enters and leaves the car's corridor (2 m either side of the path line).
- **Car window:** from the car's speed, when the car's front reaches that spot and when its rear has passed it. A stopped car is assumed to move at least 3 m/s, so it does not wait for pedestrians who will be long gone when it starts.
- The object is **relevant** only if the windows overlap, with a safety margin of 1.5 s, within the next 6 s and within 50 m ahead.

3.9 Fusion and Slow-Down Policy

Both sources pass through the same path check, and the policy combines the relevant objects:

Alert level	Condition	Action
DANGER	The car's own camera sees a relevant road user	Brake just hard enough to stop 2 m before it: required deceleration $v^2/(2 \cdot gap)$, brake command between 0.3 and 1.0. Labelled "camera+RSU" when the RSU reports the same object within 2.5 m, otherwise "camera"
CAUTION	Only the RSU reports a relevant road user	Keep driving but slow down to 15 km/h; brake directly only if the required brake command exceeds 0.3
NONE	Nothing relevant	Normal driving at 30 km/h, after 1.5 s without any threat

Table 3.4: Alert levels and actions.

The RSU warning is a **pre-emptive slow-down**, not an emergency stop: it brings the speed down while the road user is still hidden, so that the car can stop comfortably if it appears. In the baseline runs RSU messages are still received and evaluated (to measure when the warning would have arrived), but the policy ignores them.

3.10 Alert Display

The car's camera window shows a coloured banner (green, orange or red) with the alert level, the road user's type and distance, the source and the age of the information, for example "*CAUTION: HIDDEN PEDESTRIAN 6.1 m AHEAD (RSU, 160 ms old) – SLOWING TO 15 km/h*" (Figure 3.2), together with the current speed and brake command. A circle marks the position the RSU reports for the hidden road user, projected into the car's image, even while it is still behind the van. The same position is also drawn in the CARLA world, and the RSU window shows its detections, their world coordinates and the message number and size.



(a) Car camera. The CAUTION banner gives the type, distance, source and age of the warning. The orange circles mark the positions reported by the RSU for pedestrians still hidden behind the van.



(b) RSU camera: detections with their world coordinates; the header shows the number of road users, the message number and its size.



(c) CARLA view behind the car, with the mode and alert level above it and the RSU-reported positions marked in yellow.

Figure 3.2: The three windows during an assisted run of Scenario 3 (group in heavy rain).

3.11 Driving

The car is driven by a simple controller written for this project rather than CARLA's Traffic Manager autopilot. In synchronous mode the autopilot applied no throttle in our setup, and its behaviour also varies between runs. The controller steers with **pure pursuit** [7] towards a point on the lane centre a few metres ahead (from the map's waypoints; at junctions it takes the straightest continuation) and sets throttle and brake proportionally to the difference between the current and the target speed. It ignores traffic lights and other road users, so every reaction to a pedestrian or cyclist in the experiments comes from the policy above. Bicycles are moved along a straight line at constant speed (physics off), because physically simulated bicycles tipped over or were pushed off course.

CHAPTER 4: TECHNOLOGIES USED

- **CARLA simulator** (Town03, synchronous mode, weather presets) and its **Python API** for actors, sensors (RGB cameras, collision sensor), map waypoints and debug drawing [8].
- **Ultralytics YOLOv8n** with **PyTorch** for pedestrian and cyclist detection (CPU or GPU).
- **NumPy** for image buffers and the camera geometry.
- **OpenCV** for the camera windows, bounding boxes and the alert banner.
- **JSON** for the message encoding; **paho-mqtt** (MQTT client) and **aMQTT** (MQTT broker, installed with pip) for the transport.
- **pandas** and **matplotlib** for the result tables, CSV logs and figures.

CHAPTER 5: TESTING AND RESULTS

5.1 Experimental Design

Three occlusion scenarios were run. Each scenario was run twice, as **baseline** (car camera only) and **assisted** (car camera and the RSU warning received over MQTT), with everything else identical: spawn points, start times, detector, thresholds and driver. The difference between the two runs of a scenario is therefore caused only by the RSU warning.

ID	Occluder	Road user(s)	Weather	Start delay
S1	Parked van	1 pedestrian, 3.5 ± 0.3 m/s	Clear noon	0.8 s
S2	Parked truck	1 cyclist, 4.5 m/s	Clear noon	1.4 s
S3	Parked van	15 pedestrians in a 3-column grid, 3.5 ± 0.3 m/s	Heavy rain, sunset	1.5 s

Table 5.1: Occlusion scenarios.

The start delay is the time after the car starts at which the road users start moving. It was chosen so that they reach the car’s lane when the car arrives: the car reaches the crossing about 4.5 s after starting, a pedestrian needs $12 \text{ m} / 3.5 \text{ m/s} \approx 3.4 \text{ s}$ to reach the lane centre, and the cyclist $13 \text{ m} / 4.5 \text{ m/s} \approx 2.9 \text{ s}$. With later start times the car passed before the pedestrian reached the road, and there was no conflict to compare.

5.2 Metrics

Metric	Definition
RSU warning before the crossing	Time (and distance) between the first relevant RSU warning available at the car and the moment the car would reach the crossing without braking; taken from the baseline run
Warning lead time over the car camera	Time between the first relevant RSU warning and the first relevant detection by the car’s own camera (baseline run), or a note that the camera never saw the road user in time
Reaction	Who triggered the first reaction (camera or RSU) and how far before the crossing
Speed reduction	Speed at the first reaction minus the lowest speed before the crossing
Speed at closest approach	Car speed when it was closest to a road user
Minimum gap	Smallest distance between the car’s body (its bounding box) and a road user, from CARLA’s true positions
Outcome	Collision (collision sensor or gap below 0.3 m), near miss (gap below 2 m) or safe
MQTT link	Messages, losses, mean size, latency on the simulation clock, time in the channel, RSU position error

Table 5.2: Evaluation metrics.

5.3 Results

	S1 pedestrian	S2 cyclist	S3 group, rain
First RSU warning / first car-camera detection (baseline)	3.24 s / never	2.04 s / 4.04 s	2.44 s / never
RSU warning before the car would reach the crossing	1.24 s (9.7 m)	2.41 s (19.4 m)	2.16 s (17.2 m)
Warning lead time over the car camera	camera never saw it in time	2.0 s	camera never saw it in time
First reaction, baseline / assisted	none / RSU	camera / RSU	none / RSU
Reaction distance to the crossing, baseline / assisted	– / 13.0 m	3.3 m / 19.4 m	– / 17.2 m
Speed reduction, baseline / assisted	0 / 28.9 km/h	28.9 / 29.1 km/h	0 / 29.2 km/h
Speed at closest approach, baseline / assisted	28.9 / 0.0 km/h	27.7 / 28.4 km/h	28.9 / 13.1 km/h
Minimum gap, baseline / assisted	1.58 m / 5.32 m	0.00 m / 7.60 m	0.76 m / 2.76 m
Outcome, baseline / assisted	near miss / safe	collision / safe	near miss / safe

Table 5.3: Results of the three scenarios, baseline and assisted.

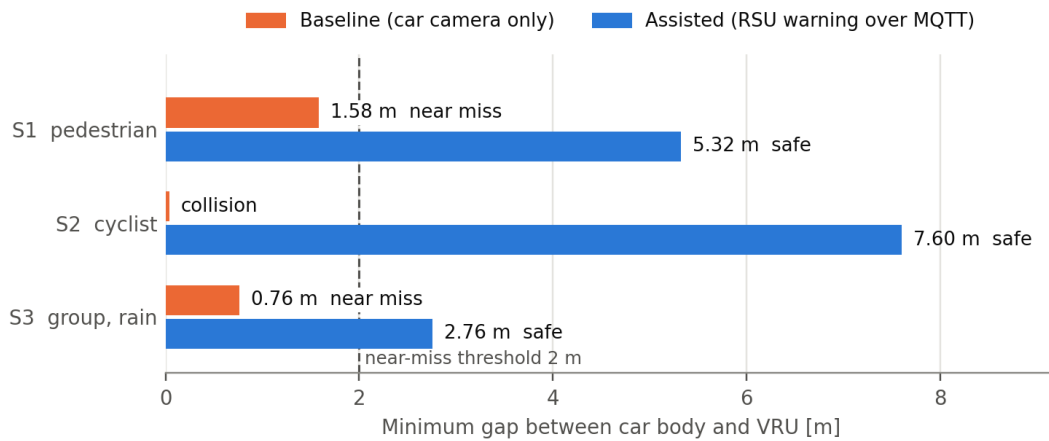


Figure 5.1: Minimum gap between the car and the pedestrian or cyclist, without and with the RSU warning.

Avoided near misses. Without the RSU warning every scenario ended in a near miss or a collision; with it every scenario ended safely, with at least 2.8 m to spare (Figure 5.1). In S1 and S3 the baseline car did not react at all: the pedestrians emerged from behind the van when the car was already level with them, outside its camera’s field of view, and it passed at full speed within 1.6 m (S1) and 0.8 m (S3) of them. In S2 the car’s camera saw the cyclist 3.3 m before the crossing, too late to stop from 28 km/h, and the car hit the cyclist. Figure 5.2 shows the car camera at comparable moments of each run.



(a) S1 baseline: no alert at 28.9 km/h; the pedestrian is hidden by the van.



(b) S1 assisted: the car has stopped on the RSU warning as the pedestrian steps out.



(c) S2 baseline: moment of contact; the car camera first saw the cyclist only 3.3 m before the crossing, too late to stop.



(d) S2 assisted: the RSU reports the hidden rider 14.2 m ahead (classified as a pedestrian in this frame).



(e) S3 baseline: no alert at 28.9 km/h, although the RSU sees 11 pedestrians (Figure 5.4).



(f) S3 assisted: CAUTION for pedestrians hidden behind the van; the car has slowed to 8.9 km/h.

Figure 5.2: The car camera without (left) and with (right) the RSU warning, at comparable moments of each scenario.

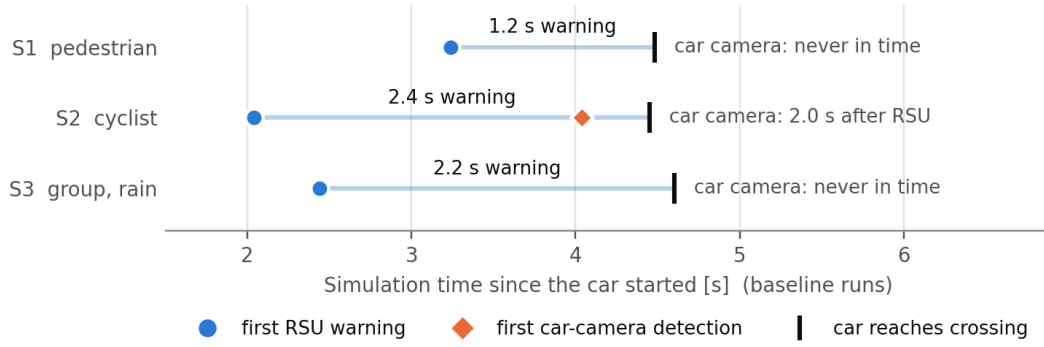


Figure 5.3: Timing in the baseline runs: first RSU warning, first detection by the car’s camera, and the time the car would have reached the crossing.

Warning lead time. The RSU warning reached the car 1.2 to 2.4 s before it would have reached the crossing, at 10 to 19 m distance (Figure 5.3). Where the car’s camera saw the road user at all (S2), the RSU was 2.0 s earlier. The S1 warning is the latest: the pedestrian starts at the edge of the pavement, and becomes relevant to the car’s path only once the car is close. Even this was enough to stop.

Speed reduction. With the RSU warning the car reacted 13 to 19 m before the crossing and slowed from about 29 km/h to a stop in all three scenarios. After stopping, the car waited until the road users had cleared its path and drove on. In S1 it was still stationary at its closest approach. In S2 its closest approach to the cyclist (7.6 m) came after it had accelerated again, which is why the speed there is 28.4 km/h; in S3 it was pulling away at 13 km/h when it passed closest to the group, 2.8 m away. Because the warnings arrived less than 20 m before the crossing at about 29 km/h, the deceleration needed to stop in time soon exceeded the threshold for a gentle slow-down, so the policy braked to a stop rather than only slowing to 15 km/h. The CAUTION banner always reads “slowing to 15 km/h”, regardless of how strongly the car brakes; the actual braking is shown by the speed and brake values below it.

5.4 MQTT Link Performance

	S1	S2	S3
Transport	MQTT	MQTT	MQTT
Messages delivered / lost	58 / 0	48 / 0	67 / 0
Mean message size	99 bytes	84 bytes	427 bytes
Latency, capture to use (simulation clock)	160 ms	160 ms	160 ms
MQTT delivery time (wall clock)	1.48 ms	1.50 ms	1.50 ms
RSU position error vs true position	0.74 m	2.14 m	0.74 m

Table 5.4: MQTT link performance and RSU localisation accuracy.

The latency of 160 ms is the 100 ms emulated delay plus about 60 ms for the age of the camera frame when it was processed and the 40 ms simulation step. Delivery through the MQTT broker took about 1.5 ms per message, negligible compared with the emulated radio delay. The mean message sizes show that the RSU saw the road users in most cycles (an empty message is 53 bytes); S3 messages are larger because they carry several pedestrians.

The RSU’s position estimates were within 0.74 m of the true position for pedestrians, which validates the pixel-to-world conversion. For the cyclist the error was 2.1 m: the box of a bicycle with its rider is



Figure 5.4: RSU camera in Scenario 3: 11 of the 15 pedestrians are tracked and sent in a single 836-byte message. An empty message is about 53 bytes and one road user about 123 bytes.

wider and its bottom edge less clearly defined than a pedestrian's feet. This is similar in size to the 2 m corridor half-width and well within the 1.5 s timing margin of the path check.

5.5 Limitations of the Evaluation

- **Single runs:** each scenario and mode was run once. Pedestrian speeds are randomised (3.5 ± 0.3 m/s), so repeated runs with different seeds would be needed for statistical statements.
- **Tuned timing:** the start delays were chosen to create a conflict. This is intended: a scenario in which the road user and the car never meet cannot show a difference between the two modes.
- **One kind of occlusion:** all three scenarios use a vehicle parked at the kerb as the occluder; they differ in the occluding vehicle, the road user type, the group size and the weather. Occlusion by a building corner at a junction was not tested.
- **Emulated radio:** MQTT runs on one computer, and the radio link is represented by a fixed 100 ms delay with no loss in the reported runs; jitter, congestion and radio range are not modelled.
- **Simple vehicle model:** our driver has a fixed cruise speed of 30 km/h; higher speeds would need earlier warnings.

CHAPTER 6: CONCLUSIONS

6.1 Summary

The project implements a complete roadside assistant in CARLA: an RSU camera placed to cover the hidden part of a crossing, YOLOv8-based detection of pedestrians and cyclists, conversion to world coordinates, tracking, a compact warning message stamped with a shared simulation clock, MQTT transport with an emulated radio delay, and a vehicle side that checks each road user against its own path, fuses the RSU information with its camera, shows a credible alert and slows down. In three occlusion scenarios, the RSU warning arrived 1.2 to 2.4 s before the crossing and turned every near miss or collision into a safe outcome, while the car's own camera alone either never saw the road user in time or saw it too late to stop.

6.2 Limitations

- Positions come from a flat-ground assumption; on slopes or for partly occluded boxes whose bottom edge is not at the feet, the estimate is less accurate.
- Fusion is rule-based: the car combines the two sources through the same path check and the policy, rather than estimating one fused state per road user.
- The pre-trained COCO detector was not fine-tuned on CARLA images, and the RSU needs lower confidence thresholds for people seen from above.
- The radio link is emulated by a fixed delay, and the experiments are single runs (Section 5.5).

6.3 Future Work

- **More occlusion types:** a building corner at a junction, a bus at a stop, and several RSUs.
- **Proper sensor fusion:** a Kalman filter that fuses RSU and car-camera observations into one track per road user, with uncertainty that grows with message age.
- **Standard messages and a realistic channel:** encode the objects as ETSI CPM, and add jitter, packet loss and range limits.
- **Statistical evaluation:** repeated runs with randomised speeds, start times and car speeds, reporting means and spread.
- **Detector:** fine-tuning on CARLA images seen from the RSU's elevated viewpoint.

BIBLIOGRAPHY

- [1] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez and V. Koltun. CARLA: An Open Urban Driving Simulator. *Proceedings of the 1st Annual Conference on Robot Learning*, 2017.
- [2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] G. Jocher, A. Chaurasia and J. Qiu. Ultralytics YOLOv8, 2023. <https://github.com/ultralytics/ultralytics>
- [4] T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV)*, 2014.
- [5] ETSI TS 103 324. Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Collective Perception Service.
- [6] ETSI EN 302 637-2. Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Cooperative Awareness Basic Service.
- [7] R. C. Coulter. Implementation of the Pure Pursuit Path Tracking Algorithm. Carnegie Mellon University, Robotics Institute, CMU-RI-TR-92-01, 1992.
- [8] CARLA Documentation. <https://carla.readthedocs.io/en/latest/>